AI ENGINEERING COURSE

Module 6

Types of Language Models

5 lessons · Personal offline study copy

Personal offline study copy. Content © Outcome School / Amit Shekhar. Not for redistribution. See study/NOTICE.md.

Contents

Module 6: Types of Language Models

- 6.1 What are Small Language Models (SLMs) and When Should We Use Them?
- 6.2 What are Large Reasoning Models (LRMs) and How Are They Different from LLMs?
- 6.3 What are Recursive Language Models (RLMs) and How Do They Work?
- 6.4 How do Diffusion Language Models (DLMs) work?
- 6.5 Jev and System One Models Explained

MODULE 6

Types of Language Models

5 lessons · ~1 min overview

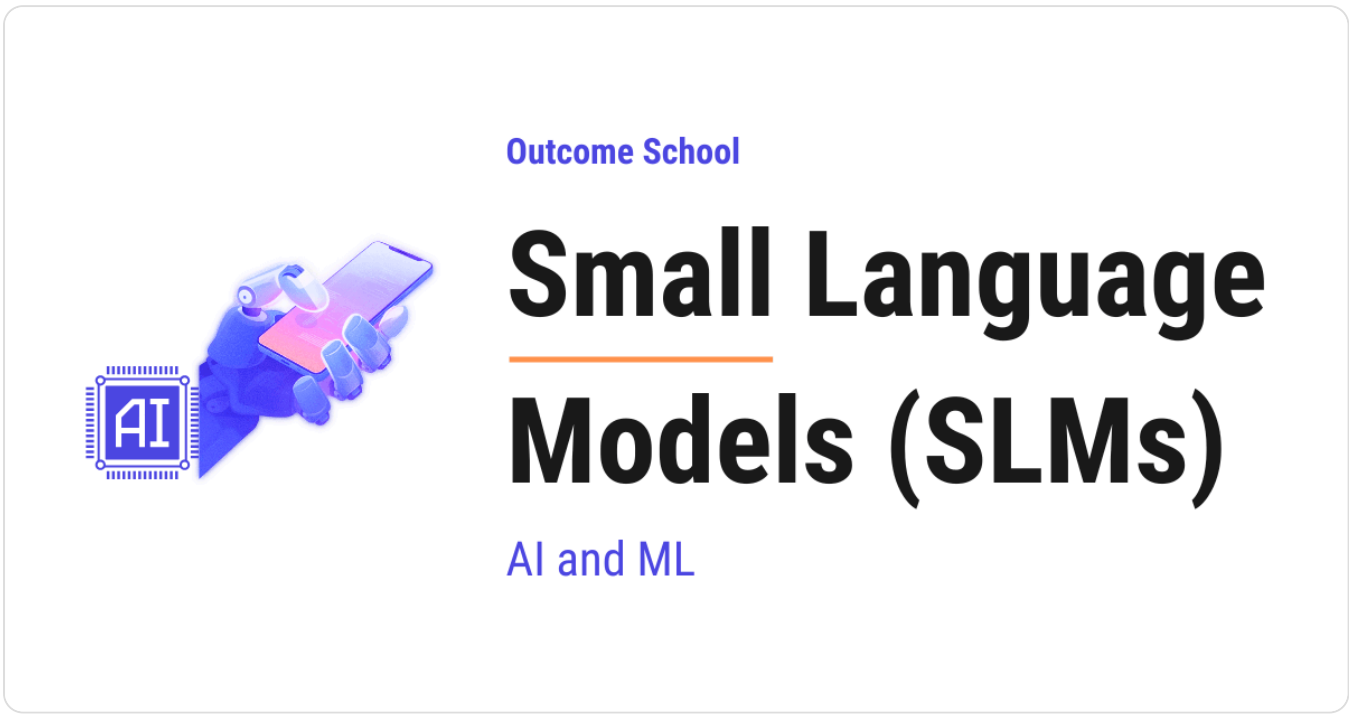
In this module, we will learn that not every language model is a large, text-generating LLM. We will see the smaller, reasoning, recursive, diffusion-based, and decision-only models and when to use which one.

By the end of this module, we will know which type of model to pick for a given problem.

LESSON 6.1

What are Small Language Models (SLMs) and When Should We Use Them?

Lesson 6.1 · ~19 min · Published May 11, 2026 · https://outcomeschool.com/blog/small-language-models-slms



A Small Language Model (SLM) is a language model that typically has less than 10 billion parameters. It is cheaper and faster than a Large Language Model, and it can even run on our own devices.

In this blog, we will learn about Small Language Models (SLMs), what counts as small, why they matter, where they shine, and the trade-offs we must keep in mind.

We will cover the following:

- SLM = Small + Language Model
- What is a Language Model?
- What Counts as "Small"?
- Popular SLMs we should know
- How SLMs Stay Capable Despite Being Small
- Why SLMs Matter
- SLM vs LLM
- The Size Spectrum
- Where SLMs Shine - Use Cases
- Trade-offs of SLMs
- When to Pick an SLM

- Quick Summary

Let's get started.

SLM = Small + Language Model

A **Small Language Model (SLM)** is a language model that is small enough to run cheaply, quickly, and even on our own laptop or phone, while still being good enough for many real tasks.

Let's decompose the name:

SLM = Small + Language Model.

- **Small** - the model has very few parameters compared to a large model. Typically less than `10` billion parameters, and often as small as `0.5` billion or `1` billion.
- **Language Model** - a neural network that learns to predict the next word in a sequence.

So, when we say "Small Language Model", we are simply saying "a language model with a small number of parameters". An SLM is a deliberate design choice. We trade some general knowledge and heavy reasoning power for speed, low cost, and the ability to run on a laptop or phone.

Now, the question is: what is a parameter? And what is a language model exactly? Let's understand both before we move forward.

What is a Language Model?

A **Language Model** is a neural network trained to predict the next token (i.e. the next small chunk of text) based on the previous tokens.

A token is roughly a small piece of text - sometimes a full word, sometimes part of a word. For example, the sentence "I love AI" might be split into `3` tokens: `I`, `love`, `AI`. The model works on tokens, not characters or words directly.

Let's say we give the model the text:

```
The sky is
```

The model predicts the next token. It might predict `blue`. Then we feed the new sequence back in and it predicts the next token, and so on. This is how the model writes full sentences, paragraphs, and even code.

The "knowledge" of the model is stored inside its **parameters**. A parameter is just a number that the model has learned during training. More parameters means more capacity to memorize patterns, facts, and reasoning shortcuts.

For example, a `1B` model has `1` billion (`1,000,000,000`) such numbers. A `70B` model has `70` billion of them. That is a `70x` jump in capacity.

So, a Language Model = a neural network + a huge number of learned parameters + the goal of predicting the next token.

Now that we know what a language model is, let's understand what makes one "small".

What Counts as "Small"?

There is no strict rule, but in practice:

- **Small Language Model (SLM)** - typically **less than 10 billion parameters**. Often `0.5B`, `1B`, `2B`, `3B`, `4B`, or up to about `9B`.
- **Large Language Model (LLM)** - typically `30B` and above. Frontier LLMs are often `70B`, `400B`, or even larger.

So, the line is roughly drawn at `10B` parameters. Anything well below that is comfortably an SLM. Anything well above that is comfortably an LLM. The middle zone (around `10B` to `30B`) is fuzzy and depends on whom we ask.

Note: "Small" is a moving target. A `7B` model was considered very small a few years back. A few years from now, today's `1B` model may feel like the new midsize. The key idea is relative size, not a fixed number.

To make this concrete, let's see real examples.

Popular SLMs we should know

Here are some well-known SLMs we should know about:

Model	Size	Developer	Notes
Phi-4-mini	3.8B	Microsoft	Strong reasoning, `128K` context
Gemma 4 E2B	2B (effective)	Google	Built for phones; `128K` context, text, image, and audio input
Gemma 4 E4B	4B (effective)	Google	Built for edge devices; `128K` context, audio input
Llama 3.2 1B	1B	Meta	Designed for mobile and edge, `128K` context
Llama 3.2 3B	3B	Meta	Stronger on-device option, `128K` context
Qwen 3.5 0.8B	0.8B	Alibaba	Among the smallest usable models, `256K` context
Qwen 3.5 2B	2B	Alibaba	Multimodal, thinking-mode toggle, `256K` context
Qwen 3.5 4B	4B	Alibaba	Multimodal, `256K` context
Qwen 3.5 9B	9B	Alibaba	Sweet spot of small series, rivals older `30B` models, `256K` context
SmolLM3 3B	3B	Hugging Face	Open instruct and reasoning, `128K` context

Note: The "E" in Gemma 4 E2B and E4B stands for "Effective" parameters. Gemma 4 uses Per-Layer Embeddings, so the parameter count that matters at inference time is smaller than the total parameter count.

Here, we can see that the parameter counts range from `0.8B` to `9B`. The smaller ones can run on a phone, while the larger ones run on a laptop with a decent GPU. Compare this to a `70B` LLM, which needs a heavy GPU server to run at all.

Now, a natural question arises - how can a `3B` model be useful at all when a `70B` model has so much more capacity? Let's understand.

How SLMs Stay Capable Despite Being Small

A few years back, a `3B` model was very weak. Today, a `3B` SLM can be surprisingly capable. So, what changed? I am sure many of us must be knowing the headline answer - better data, better training, better architecture. Let's see all three:

1. Better training data.

Modern SLMs are trained on carefully filtered, very high-quality data. Instead of dumping the whole internet into the training pipeline, teams now filter, clean, and pick the best `1` to `15` trillion tokens. Quality beats quantity.

2. Better training techniques.

Techniques like **knowledge distillation** (where a small "student" model learns from a large "teacher" model), better optimizers, and longer training runs squeeze more capability out of every parameter.

3. Smarter architecture choices.

Choices like Grouped-Query Attention, RoPE (Rotary Position Embedding), and other recent ideas make small models faster and stronger without needing more parameters. If we want to go deeper, we have a detailed blog on Grouped Query Attention that explains one such technique.

So, a modern `3B` SLM today is far stronger than a `3B` model from a few years ago, thanks to better data and better training.

Now, let's understand why we even care about SLMs.

Why SLMs Matter

Now, the question is: why do we need SLMs when LLMs are so powerful? The answer is: SLMs win on cost, speed, privacy, and on-device deployment.

Let's go through each reason.

1. Lower cost.

LLMs are expensive to run. Every token we generate costs money, either in API (Application Programming Interface, i.e. the cloud service we call to use the model) charges or in GPU (Graphics Processing Unit) time. SLMs cost a tiny fraction of that.

Let's put this into perspective with real numbers. For a typical workload of `1` million output tokens:

- A frontier LLM might cost around `$25` per million output tokens via an API.
- A `3B` SLM running on our own GPU might cost around `$0.10` to `$0.20` per million output tokens in electricity and hardware time.

That is roughly a `100x` to `200x` cost reduction. For a product that processes `100` million tokens per day, this is the difference between `$2,500` per day and around `$15` per day.

2. Lower latency.

Smaller models respond faster. Less computation per token means less time per token.

For the same prompt, the **time to first token** (i.e. how long before the first word appears) looks like:

- A `70B` model on a single GPU might take around `2` seconds.
- A `3B` SLM on the same hardware might take around `100` milliseconds (`0.1` second).

That is a `20x` speedup. For a chat or voice assistant, that difference is the gap between "feels slow" and "feels instant".

3. Privacy.

SLMs are small enough to run on our own machine. No data leaves our laptop or our server. This matters for medical records, financial data, internal company documents, and any case where sending data to a cloud API is not allowed.

We have a detailed blog on how Ollama works that explains step by step how we run a model on our own computer.

4. On-device and edge deployment.

A `3B` model can run on a modern phone. A `1B` model can run on a small laptop CPU. A `0.5B` model can run on tiny edge devices. This opens up offline AI - apps that work without internet, on the user's own device. For example, a smart keyboard that suggests the next sentence, a note-taking app that summarizes our notes locally, or a translator that works on a long flight without WiFi. These on-device models are usually packaged in the GGUF format and run through an engine like llama.cpp. We have a detailed blog on how GGUF works that explains the single-file format these models run from.

5. Easier fine-tuning.

Fine-tuning means we take a pre-trained model and train it a little more on our own data so that it gets better at our specific task. Fine-tuning a `70B` model needs a small data center. Fine-tuning a `3B` model can be done on a single consumer GPU in a few hours. So, teams can build their own custom models for their specific domain very easily.

6. Reliability and control.

There is no API rate limit when we run an SLM ourselves. No outages from a third party. No surprise price changes. We own the model and we own the deployment.

So, SLMs open up use cases that LLMs cannot reach at all - on-device, fully offline, fully private, and fully under our control.

Now, let's compare SLMs and LLMs side by side.

SLM vs LLM

Let me tabulate the differences between SLMs and LLMs for your better understanding so that you can decide which one to use based on your use case.

Aspect	SLM	LLM
Parameters	`0.5B` to `~10B`	`30B` to `400B+`
Memory needed	`1` to `18 GB`	`60` to `800+ GB`
Hardware	Phone, laptop, single GPU	Multi-GPU server
Latency (first token)	`~50-200 ms`	`~500 ms - 3 s`
Cost per `1M` output tokens	`~$0.10 - $1`	`~$5 - $30`
General knowledge	Decent, but limited	Very broad
Complex reasoning	Weak to medium	Strong
Long context quality	Decent at `128K`	Strong at `1M+`
Privacy	Easy (run locally)	Hard (cloud APIs)
Fine-tuning effort	Easy and cheap	Hard and expensive

Here, we can see that each side has its own strengths. SLMs win in deployment, cost, latency, and privacy. LLMs win in general knowledge, reasoning depth, and handling very long contexts.

There is no clear winner here. It depends on our use case.

Let's put this into perspective with a small example. Suppose we want to summarize a paragraph of text into one line:

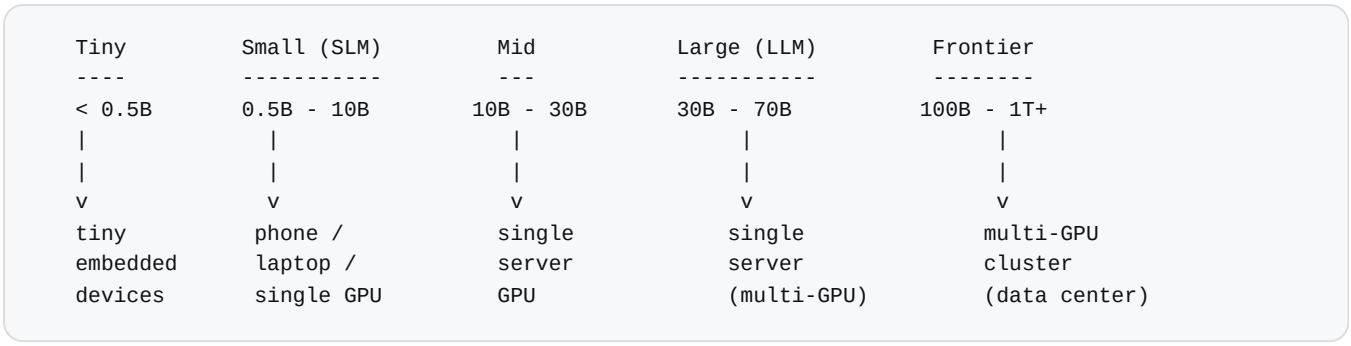
- With a `1B` SLM: takes around `100 ms`, uses around `2 GB` of memory, costs almost nothing.
- With a `70B` LLM: takes around `2` seconds, uses around `140 GB` of memory, costs around `$0.0002` per call via API.

For one user, both are fine. For `1` million calls per day, the SLM saves us thousands of dollars and gives a much better user experience. I have used round numbers here for the sake of understanding, the real numbers depend on the exact hardware and the exact API price.

Now, let's visualize where SLMs sit on the size spectrum.

The Size Spectrum

Here is a simple diagram showing where SLMs fit:



Here, we can see five rough buckets along the size axis:

- **Tiny** (`<0.5B`) - very small models for embedded use.
- **Small / SLM** (`0.5B` to `~10B`) - the SLM zone. Smaller ones run on a phone; larger ones on a laptop or single GPU.
- **Mid** (`10B` to `30B`) - the fuzzy middle between SLM and LLM. Could go either way.
- **Large / LLM** (`30B` to `70B`) - serious GPU servers needed.
- **Frontier** (`100B` and beyond) - large clusters needed.

The SLM bucket is the sweet spot for most production use cases that do not need deep reasoning. It is small enough to deploy easily and large enough to do useful work.

Now, let's see where SLMs really shine in practice.

Where SLMs Shine - Use Cases

SLMs are the right tool for many real tasks. Let's go through the most common ones.

1. On-device assistants.

A `1B` to `3B` SLM can run on a phone or laptop. So, we can build a writing assistant, a code helper, or a chat assistant that works offline. No internet, no cloud, no API key. The model lives inside the app.

For example, a `1B` model uses around `2 GB` of memory and can produce around `30` tokens per second on a modern phone. That is fast enough for real-time chat.

2. Classification.

Many real tasks are simply: "given this input, pick one label out of N categories". For example, classifying a support ticket as `billing`, `technical`, or `account`. An SLM, especially after a small fine-tune, handles this very well.

A `0.5B` fine-tuned model can classify thousands of inputs per second on a single CPU. For high-volume classification, this is hard to beat.

A newer option for this exact job is a System One Model like Jev, which does not generate text at all and returns the label with a calibrated probability in a single pass.

3. Structured extraction.

Pulling structured data out of unstructured text. For example, given an email, extract the sender, the date, the action item, and the deadline. SLMs do this very well, especially when we use constrained decoding (i.e. forcing the model to produce valid JSON).

4. Agent task heads.

In a larger agent system, many small steps do not need a heavy reasoning model. Routing a request, selecting a tool from a list, formatting an output - all of these can be done by an SLM. The big LLM is only called when complex reasoning is actually needed.

If we want to go deeper into how agents work end to end, we can read AI Agent.

5. Fine-tuned domain models.

Suppose we work in a niche field - for example, a specific style of legal contracts in our country. Fine-tuning a `3B` SLM on `10,000` examples of our domain often beats a generic `70B` LLM on that specific task. We get a model that is small, fast, cheap, and very accurate within our domain.

6. High-volume pipelines.

Imagine processing `10` million customer reviews per day to extract sentiment. An LLM API would cost around `$10,000` per day. A self-hosted SLM might cost around `$100` per day for the same job. The math speaks for itself.

So, SLMs shine wherever the task is narrow, latency-sensitive, or privacy-sensitive.

Now, let's be honest about where SLMs do not shine.

Trade-offs of SLMs

SLMs are great, but they are not magic. Let's understand the trade-offs.

1. Less general knowledge.

A `1B` model has `70x` fewer parameters than a `70B` model. So, it has memorized far fewer facts about the world. Ask it about a niche historical event or a rare scientific concept, and it may simply not know.

2. Weaker complex reasoning.

For multi-step reasoning - long math chains or complex planning - SLMs often struggle. They may give wrong answers with high confidence.

3. Weaker long-context quality.

Most modern SLMs already support `128K` tokens or more, so the size of the context window is no longer the main limit. The real gap is in **how well** the model actually uses that window. For tasks like

needle-in-haystack search across `100K` tokens, or long multi-document reasoning, a `3B` SLM is meaningfully weaker than a `200B+` frontier LLM, even when both technically support the same window size. We have a detailed blog on Recursive Language Models (RLMs) that sidesteps this entirely by splitting a long input into small pieces and letting an SLM-sized model work on one piece at a time.

4. More fragile.

SLMs can be more sensitive to prompt wording. A small change in phrasing can lead to a very different answer. So, we often need careful prompting or fine-tuning to get reliable behavior.

5. Less polished writing.

For long-form creative writing, marketing copy, or careful multi-step reasoning, the gap between an SLM and a top LLM is real. The SLM may sound less natural and less coherent.

This trade-off - **less reasoning power and general knowledge, more speed and lower cost** - is the core deal we are making when we choose an SLM.

Note: The gap between SLMs and LLMs keeps shrinking with every new generation. A `3B` SLM today often matches or beats a `13B` model from two years ago. So, the trade-off list above is correct today, but the gap will keep getting smaller.

Now, the natural question is: when do we pick an SLM?

When to Pick an SLM

Based on our use case, here is a simple decision guide:

Pick an SLM when:

- The task is narrow and well-defined (classification, extraction, routing).
- We expect very high volume (millions of calls per day).
- We need very low latency (under `200 ms`).
- The data is sensitive and must stay on our own server or device.
- We need offline / on-device deployment.
- We have training data and can fine-tune for our domain.

Pick an LLM when:

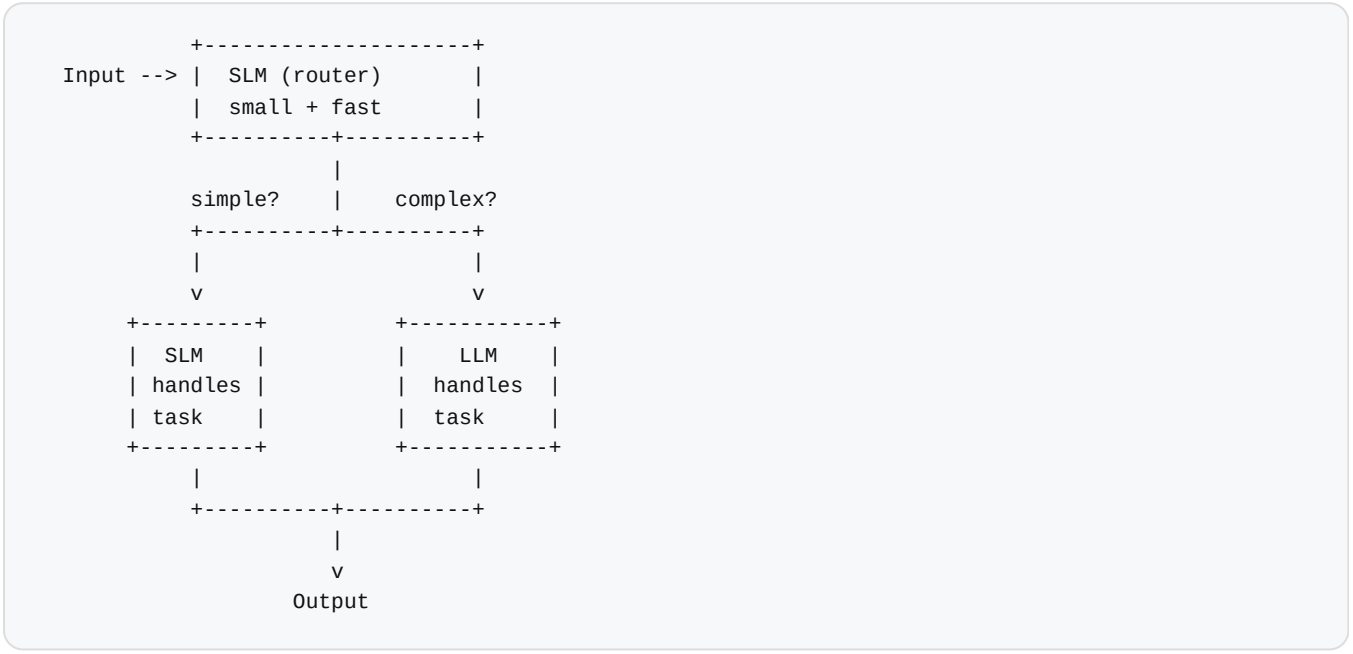
- The task needs deep reasoning over multiple steps.
- The task needs broad world knowledge.
- The task needs very long context (i.e. large documents or long histories).
- The volume is low enough that API cost is not a concern.
- We need the highest possible quality and we are willing to pay for it.

Pick a hybrid (SLM + LLM) when:

- Most steps in our workflow are simple, but a few need heavy reasoning.

This is where the hybrid setup comes into the picture. The SLM handles routing, classification, extraction, and tool-selection, while the LLM is called only for the hard reasoning steps. This gives us the speed and cost of an SLM with the depth of an LLM where it really matters.

Here is a simple diagram of the hybrid pattern:



Here, we can see that the SLM acts as the front-line router. For simple tasks (let's say `90%` of traffic), the SLM handles the work end to end. For complex tasks (the remaining `10%`), the LLM is called. So, we pay LLM cost on only `10%` of the traffic, while still getting LLM-quality answers when we need them. This pattern is a form of LLM Routing, where each query is sent to the right model based on cost, latency, and quality.

I will highly recommend starting with an SLM whenever the task allows it. We get faster iteration, lower cost, and full control. We can always upgrade to an LLM later if the task truly needs it.

Quick Summary

Let's recap what we have learned:

- **SLM = Small + Language Model.** A language model with a small number of parameters, typically less than `10` billion.
- **Language Model.** A neural network that predicts the next token given the previous tokens. The "knowledge" is stored in its parameters.
- **Popular SLMs we should know.** Phi-4-mini (`3.8B`), Gemma 4 (`E2B` / `E4B`), Llama 3.2 (`1B` / `3B`), Qwen 3.5 (`0.8B` / `2B` / `4B` / `9B`), SmolLM3 (`3B`).

- **Why modern SLMs are strong.** Better training data, knowledge distillation from larger teacher models, and smarter architecture choices.
- **Why SLMs matter.** Lower cost, lower latency, privacy, on-device deployment, easier fine-tuning, and full control.
- **Cost example.** A `3B` SLM can be around `100x` to `200x` cheaper per million tokens than a frontier LLM.
- **Latency example.** A `3B` SLM responds in around `100 ms` while a `70B` LLM may take around `2` seconds.
- **Memory example.** A `1B` SLM needs around `2 GB` while a `70B` LLM needs around `140 GB`.
- **Where SLMs shine.** On-device assistants, classification, structured extraction, agent task heads, fine-tuned domain models, and high-volume pipelines.
- **Trade-offs.** Less general knowledge, weaker complex reasoning, weaker long-context quality, more sensitive to prompt wording.
- **When to pick an SLM.** Narrow tasks, high volume, low latency, privacy needs, on-device, or when we have data to fine-tune.
- **Hybrid setup.** SLM for routine steps, LLM for the few hard reasoning steps. Best of both worlds, based on our use case.

Now, we have understood Small Language Models (SLMs).

Frequently Asked Questions

Can a Small Language Model beat a Large Language Model?

Yes, on a narrow task. Fine-tuning a `3B` SLM on `10,000` examples of our domain often beats a generic `70B` LLM on that specific task. We get a model that is small, fast, cheap, and very accurate within our domain, though it still has less general knowledge.

Can an SLM run on a phone?

Yes. A `3B` model can run on a modern phone, and a `1B` model can run on a small laptop CPU. For example, a `1B` model uses around `2 GB` of memory and can produce around `30` tokens per second on a modern phone, which is fast enough for real-time chat.

Do SLMs support long context?

Yes. Most modern SLMs already support `128K` tokens or more. The real gap is in how well the model uses that window. For tasks like needle-in-haystack search across `100K` tokens, a `3B` SLM is meaningfully weaker than a `200B+` frontier LLM.

Will the gap between SLMs and LLMs keep shrinking?

Yes. The gap keeps shrinking with every new generation. A `3B` SLM today often matches or beats a `13B` model from two years ago.

Are SLMs sensitive to how we write the prompt?

Yes. SLMs can be more sensitive to prompt wording, and a small change in phrasing can lead to a very different answer. So, we often need careful prompting or fine-tuning to get reliable behavior from an SLM.

LESSON 6.2

What are Large Reasoning Models (LRMs) and How Are They Different from LLMs?

Lesson 6.2 · ~20 min · Published May 12, 2026 · https://outcomeschool.com/blog/large-reasoning-models



A Large Reasoning Model (LRM) is a large language model that does not jump straight to the answer. It first produces a long chain of intermediate reasoning steps, and then gives the final answer.

In this blog, we will learn about Large Reasoning Models (LRMs), how they are different from standard Large Language Models, how they think before they answer, how they are trained, and when we must use them.

We will cover the following:

- The Big Picture
- What is a Large Reasoning Model (LRM)?
- LLM vs LRM
- How does an LRM actually think?
- Test-time compute: thinking longer makes them smarter
- How are LRMs trained?
- Input and Output: training phase vs prediction phase
- When to use an LRM, and when to use a regular LLM
- Popular LRMs we should know
- Common Mistakes when using LRMs

- Quick Summary

Let's get started.

The Big Picture

Before we go further, a quick word on **tokens**. A token is a small piece of text, roughly a word or a part of a word. The model reads and writes text one token at a time. When we say "next token", we mean the next small piece of text the model produces.

A standard Large Language Model answers as soon as we ask. It predicts the next token, one after the other, and we get the reply.

A Large Reasoning Model does something different. Before it gives the final answer, it first thinks. It writes out a long internal scratchpad of steps, checks itself, sometimes changes its mind, and only then it speaks.

In simple words:

Large Reasoning Model = A Large Language Model that is trained to think first, and answer later.

What is a Large Reasoning Model (LRM)?

Let's decompose the name:

LRM = Large + Reasoning + Model

- **Large** - it has billions of parameters, often in the range of `7B` to `700B+`, just like a normal Large Language Model.
- **Reasoning** - it does not jump straight to the answer. It produces a long chain of intermediate steps first.
- **Model** - it is still a neural network that predicts the next token.

So, a Large Reasoning Model is a Large Language Model that has been specially trained to spend extra tokens thinking before it gives the final answer.

These models are also called **reasoning models** or **thinking models**. They all mean the same thing.

Examples we may have heard of: OpenAI's `GPT-5.5 Thinking`, DeepSeek `R1`, Qwen3, Google's Gemini 3 with `Deep Think`, Anthropic's `extended thinking` mode in Claude, and xAI's Grok 4.1 with `Think` mode.

LLM vs LRM

The best way to understand LRMs is to compare them with a regular LLM.

Let's say we ask both a small word problem. We will reuse this same problem throughout the blog to keep the comparisons clean.

> A shop sells pens at `2` for `5` rupees, and notebooks at `3` for `12` rupees. We buy `6` pens and `9` notebooks. How much do we pay in total?

A regular LLM may answer in `1` second, generate just a few tokens like "`51` rupees", and sometimes get the answer wrong because it tries to guess in one shot.

A Large Reasoning Model takes a different path. It first thinks - working out the per-unit cost, multiplying, adding, and verifying the total - and only then writes the final answer. For a small problem like this, the trace may use a few hundred reasoning tokens. For harder problems, it can stretch into thousands or tens of thousands of tokens.

The reasoning tokens are usually hidden from the end user. We only see the final answer. But the model paid the cost of thinking, and that is why the answer is more accurate.

A quick visual of what each one is doing:

```
LLM:   [question] ---> [short answer]           (fast, sometimes wrong)
LRM:   [question] ---> [long thinking trace]
                       ---> [short answer]     (slow, more often right)
```

Here, we can see that the LRM adds a thinking step in the middle, which the LLM skips entirely.

Let me tabulate the differences between LLM and LRM for your better understanding so that you can decide which one to use based on your use case.

Aspect	Standard LLM	Large Reasoning Model (LRM)
Answering style	Direct, one shot	Thinks first, answers later
Tokens used	Few (`100` to `500`)	Many (`5,000` to `50,000`)
Latency	`1` to `3` seconds	`10` to `120` seconds
Cost per query	Low	`10x` to `30x` higher
Strength	Simple chat, summaries, drafting	Math, code, logic, planning
Weakness	Hard multi-step problems	Slow and expensive for easy tasks

Both are useful, just for different jobs.

How does an LRM actually think?

This was all about the difference between LLMs and LRMs. Now, let's understand what really happens inside an LRM when it thinks.

The thinking inside an LRM is just more text. There is no magic new component. The model writes out its thought process as tokens, the same way it writes any other tokens.

This long block of internal text is often called the **reasoning trace** or **thinking trace**. For our pen and notebook question above, it looks somewhat like below:

```
<think>
Pens: 2 pens cost 5 rupees, so 1 pen costs 2.5 rupees.
6 pens cost 6 * 2.5 = 15 rupees.
Notebooks: 3 notebooks cost 12 rupees, so 1 notebook costs 4 rupees.
9 notebooks cost 9 * 4 = 36 rupees.
Total = 15 + 36 = 51 rupees.
Let me verify. 6 pens means 3 packs of 2 pens, each pack 5 rupees, so 3 * 5 = 15. Correct.
9 notebooks means 3 packs of 3 notebooks, each pack 12 rupees, so 3 * 12 = 36. Correct.
Total 15 + 36 = 51. Correct.
</think>
We pay 51 rupees in total.
```

Here, we can see three things happening:

- The model breaks the problem into smaller steps.
- It tries an approach, computes intermediate values, and writes them down.
- It checks its own answer at the end before giving the final reply.

This is the same pattern a careful student follows on a rough sheet during an exam. The rough sheet is the reasoning trace. The final answer is what the student writes on the answer sheet.

A simple flow diagram:

```
User question
     |
     v
+-------------------+
|   LRM thinking    |    <-- long internal scratchpad
|  (reasoning trace)|        (often hidden from user)
+-------------------+
     |
     v
Final short answer
```

The reasoning trace is hidden in many products (like `GPT-5.5 Thinking`, where we only see a short summary), and visible in some others (like DeepSeek `R1`).

Test-time compute: thinking longer makes them smarter

Here comes one of the most important ideas behind LRMs.

In a normal LLM, we make the model smarter by training a bigger model on more data. That is **train-time compute**. We pay once during training, and then every answer is fast.

LRMs add a second knob, called **test-time compute**. This means the model becomes smarter at the moment of answering, by thinking for longer.

Let's put this into perspective with the actual published numbers from OpenAI on the AIME 2024 math competition (a hard high school contest):

- **GPT-4o** (a regular LLM, no extended reasoning): around `12%` correct.
- **o1** (an LRM with one full reasoning attempt): around `74%` correct.
- **o1** with majority vote across `64` parallel attempts: around `83%` correct.
- **o1** re-ranking across `1,000` attempts: around `93%` correct.

The first row is a regular LLM shown for baseline. The next three rows are the same `o1` model - what changes from row to row is purely how much test-time compute we let it spend at answer time. This is why we say LRMs scale with **test-time compute**.

In our student analogy, this is like giving the same student more pages of rough sheet during an exam. Same student, same brain, but a longer rough sheet often leads to a better final answer.

We can also spend extra compute at answer time from the outside, by making a regular LLM try many reasoning paths, judge them, and keep only the good ones. We have a detailed blog on Tree of Thoughts that covers this end to end.

But, here is the catch. More thinking is not free. More tokens means more time and more cost. Doubling the thinking tokens roughly doubles the output cost of that query, and the output cost is usually most of the bill for an LRM.

This trade-off - **less speed, more accuracy** - is the core reason LRMs exist as a separate class of models.

How fast those thinking tokens actually come out is a hardware question, and we have a detailed blog on how an LPU works that explains a chip built to make exactly this part fast.

Note: Many LRM APIs let us control how much the model is allowed to think. For example, we can set a thinking budget of `1,024`, `8,192`, or `32,768` tokens. A small budget gives a fast and cheap answer. A large budget gives a slow but more accurate answer. We can tune this knob based on our use case. Newer APIs are moving toward **adaptive thinking**, where the model decides how much to think based on the complexity of the query, instead of us setting a fixed budget.

How are LRMs trained?

Now that we have understood how an LRM thinks at answer time, let's understand how it learns to think in the first place.

A regular LLM is trained in two main stages: pre-training on a huge amount of text from the internet, and instruction tuning so it learns to follow our prompts.

LRMs add one more stage on top: **reinforcement learning on reasoning tasks**.

The idea is simple:

- Give the model a hard problem with a known correct answer.
- Let it generate many long reasoning traces in parallel (a group of `16`, `32`, or `64` traces for the same problem).
- Check which traces lead to the correct final answer. Math problems have a clear right answer, so this check is easy and automatic.
- Score each trace relative to the rest of the group. The traces that did better than the group average are reinforced. The ones that did worse are penalised.
- Update the model so it becomes more likely to follow the good reasoning patterns next time.

Note: This group-relative scoring is the heart of **GRPO (Group Relative Policy Optimization)**, the algorithm used to train DeepSeek `R1`. The "compare to the group, not to an absolute target" trick is what makes RLVR stable at this scale.

This is called **Reinforcement Learning with Verifiable Rewards (RLVR)**. The word **verifiable** is important. Means, the reward comes from a simple automatic checker, not from a human opinion the way it does in Deep RL from Human Preferences. The checker can be:

- Math: does the final number match the correct answer?
- Code: does the generated code pass all unit tests?
- Logic: does the answer match the puzzle's known solution?

Because the reward is automatic, we can run this training loop millions of times without any human in the middle. In our student analogy, we are teaching the student which kinds of rough sheet patterns lead to the right final answer. We let them try many times, score each attempt, and reward the winners. Over time, the model learns useful habits like:

- Break the problem down before solving.
- Try a step, check the unit, then continue.
- Spot a mistake, go back, and try a different approach.

This is exactly what we see in the long reasoning traces.

Input and Output: training phase vs prediction phase

Now, let's put everything together by looking at exact examples of what went into the model and what came out, in both the training phase and the prediction phase, for both LLM and LRM.

Training phase: Standard LLM

In LLM training, the model is shown a piece of text with the next token hidden. It must predict that token.

Input to the model:

```
The capital of France is
```

Output from the model (the predicted next token):

```
Paris
```

The actual next token in the data is also "Paris". The model got it right, so the loss is small. If the model had predicted "London", the loss would be large, and the weights would be nudged to fix this.

Training phase: Large Reasoning Model

In LRM training, the model is shown a hard problem with a known correct answer. It generates several full attempts to solve it.

Input to the model:

```
A shop sells pens at 2 for 5 rupees, and notebooks at 3 for 12 rupees.
We buy 6 pens and 9 notebooks. How much do we pay in total?
```

Output from the model (one of the `16` attempted traces):

```
<think>
Pens: 2 pens cost 5 rupees, so 1 pen costs 2.5 rupees.
6 pens cost 6 * 2.5 = 15 rupees.
Notebooks: 3 notebooks cost 12 rupees, so 1 notebook costs 4 rupees.
9 notebooks cost 9 * 4 = 36 rupees.
Total = 15 + 36 = 51 rupees.
Let me verify. 6 pens means 3 packs of 2 pens, each pack 5 rupees, so 3 * 5 = 15. Correct.
9 notebooks means 3 packs of 3 notebooks, each pack 12 rupees, so 3 * 12 = 36. Correct.
Total 15 + 36 = 51. Correct.
</think>
We pay 51 rupees in total.
```

During training, the model is asked to produce many such attempts for the same problem - say `16` traces in parallel. A simple checker compares the final answer of each trace against the known correct

answer "51 rupees". Maybe `6` traces got `51`, and `10` got something wrong like `48` or `54`. The `6` good ones score above the group average and are reinforced. The `10` wrong ones score below the group average and are penalised. The model is then updated to produce more of the rewarding kinds of traces.

Prediction phase: Standard LLM

The model is already trained. We give it a prompt, it produces an answer.

Input to the model:

```
Write a haiku about rain.
```

Output from the model:

```
Soft drops on the leaves,
A grey sky weeps to the earth,
Spring whispers goodbye.
```

The model goes straight from prompt to answer. There is no internal thinking step. The user sees the full output.

Prediction phase: Large Reasoning Model

The model is already trained. We give it a prompt, it produces a thinking trace and then the final answer.

Input to the model:

```
A shop sells pens at 2 for 5 rupees, and notebooks at 3 for 12 rupees.
We buy 6 pens and 9 notebooks. How much do we pay in total?
```

Output from the model:

```
<think>
Pens: 2 pens cost 5 rupees, so 1 pen costs 2.5 rupees.
6 pens cost 6 * 2.5 = 15 rupees.
Notebooks: 3 notebooks cost 12 rupees, so 1 notebook costs 4 rupees.
9 notebooks cost 9 * 4 = 36 rupees.
Total = 15 + 36 = 51 rupees.
Let me verify. 6 pens means 3 packs of 2 pens, each pack 5 rupees, so 3 * 5 = 15. Correct.
9 notebooks means 3 packs of 3 notebooks, each pack 12 rupees, so 3 * 12 = 36. Correct.
Total 15 + 36 = 51. Correct.
</think>
We pay 51 rupees in total.
```

The text inside `<think>...</think>` is the long reasoning trace. The line after it is the final answer. In many products, the user only sees "We pay 51 rupees in total." - the trace is hidden.

Notice that this is the same problem we used in the training example above. During training, this was one of `16` attempted traces that got scored against the known answer "51". During prediction, the model now produces the same kind of trace because that is exactly what training rewarded it to do.

Quick recap of the four cases

In LLM training, the input is a chunk of text and the output is the next token. The loss measures how close the predicted token is to the actual next token. In LRM training, the input is a hard problem with a known answer, and the output is a full reasoning trace ending in a final answer. The reward checks if the final answer is correct.

In LLM prediction, the input is a user prompt and the output is a direct answer. In LRM prediction, the input is a user prompt and the output is a long internal thinking trace followed by a short final answer.

When to use an LRM, and when to use a regular LLM

Now that we have seen exactly what goes in and what comes out of both models, the most practical question for any AI Engineer: when to use an LRM and when to use a regular LLM.

We use a **regular LLM** when:

- The task is short and direct.
- We want a quick chat reply, a simple summary, or a draft email.
- Latency matters more than perfect correctness.
- Cost per query must be low because we are doing millions of calls.

We use an **LRM** when:

- The task needs multiple steps of careful thought.
- A wrong answer is costly. Examples: complex math, hard coding problems, scientific questions, multi-step planning, deep data analysis, debugging tricky logic.
- We can afford to wait `30` seconds or more.
- We can afford the higher token cost.

And when the answer is not text at all but a decision, like which category, which team, or safe or not, there is a third option built just for that. We have a detailed blog on Jev and System One Models that explains how such a model returns a typed decision with a calibrated probability in a fraction of a second.

Popular LRMs we should know

Now, let's look at the popular LRMs we should know:

- **OpenAI GPT-5.5 Thinking** - the current OpenAI reasoning flagship. The raw reasoning trace is hidden, and we only see a short summary plus the final answer.
- **DeepSeek `R1` and `R1-Zero`** - open weights, shows the full reasoning trace, uses RLVR as a key training stage. `R1-Zero` is the purest RLVR demonstration, trained with only rule-based rewards on top of the base model, with no **SFT (Supervised Fine-Tuning, i.e. training on human-written examples)** in between. DeepSeek's newer line (V3.1 and V4) packs both thinking and non-thinking modes into one model.
- **Qwen3** - open weights from Alibaba. One unified model that supports both a thinking mode and a non-thinking mode, with an explicit thinking budget knob.
- **Google Gemini 3 with Deep Think** - Google's reasoning mode inside the Gemini family. It can spend tens of seconds, and sometimes minutes, of test-time compute on a truly hard problem.
- **Anthropic Claude Opus 4.7 with extended thinking** - Anthropic's reasoning mode, available as a first-class feature across the Claude 4.7 family.
- **xAI Grok 4.1 with Think mode** - xAI's reasoning model. It pairs the thinking step with DeepSearch over X and the web.

All of them share the same core idea: think first, answer later. The differences are in the size of the model, the training data, and whether the reasoning trace is shown to us or hidden.

Common Mistakes when using LRMs

Now, let's look at a few common mistakes we must avoid.

Mistake 1: Using an LRM for everything.

If we use an LRM for a simple "translate this sentence" or "summarize this paragraph", we are paying `10x` to `30x` more cost and waiting `30` seconds for a task that a normal LLM would finish in `1` second. We must route only the hard tasks to the LRM.

Mistake 2: Ignoring the cost of reasoning tokens.

Even if the reasoning trace is hidden, we still pay for it. A query that returns `200` visible tokens may have used `15,000` reasoning tokens behind the scenes. We must always check the billing model.

Mistake 3: Adding "think step by step" to an LRM prompt.

The model is already trained to think step by step. Adding extra "think carefully" instructions usually does not help, and sometimes confuses the model. Keep the prompt clean and just state the problem.

Mistake 4: Not setting a thinking budget on production traffic.

If we let an LRM think for as long as it wants, a single hard prompt can burn through tens of thousands of reasoning tokens. The cost per query swings wildly, and one bad input can dominate the day's bill. For production traffic, we must set a thinking budget - for example, `8,192` tokens for most

paths, and `32,768` only for the truly hard ones. Adaptive thinking helps, but a hard upper bound is the safer default.

Mistake 5: Treating the reasoning trace as the final answer.

The reasoning trace is a draft. It can contain wrong intermediate steps, scratch values, and corrections. We must only show the final answer to the end user, not the raw trace.

Quick Summary

Let's recap what we have learned:

- **LRM** stands for **Large Reasoning Model**. It is a Large Language Model that is trained to think first and answer later.
- **Reasoning trace** is the long internal scratchpad of tokens the model produces before the final answer.
- **Test-time compute** is the idea that we can make a model smarter by letting it think longer at answer time.
- **RLVR** (Reinforcement Learning with Verifiable Rewards) is the training trick that teaches the model to write good reasoning traces, using automatic checkers like math answers and unit tests.
- **LRMs are best for hard, multi-step problems.** For simple tasks, a regular LLM is faster and cheaper.
- **Popular LRMs**: `GPT-5.5 Thinking`, DeepSeek `R1`, Qwen3, Gemini 3 with Deep Think, Claude Opus 4.7 with extended thinking, and Grok 4.1 with Think mode.

Now, we have understood Large Reasoning Models (LRMs).

Frequently Asked Questions

Is a Large Reasoning Model a different kind of neural network from an LLM?

No. A Large Reasoning Model is still a neural network that predicts the next token, just like a Large Language Model. The thinking inside an LRM is just more text, and there is no magic new component. The difference is that it has been specially trained to spend extra tokens thinking before the final answer.

Can we see the reasoning trace of an LRM?

It depends on the product. The reasoning trace is hidden in many products, like `GPT-5.5 Thinking`, where we only see a short summary and the final answer. It is visible in some others, like DeepSeek

`R1`, which shows the full reasoning trace.

Does RLVR training need humans to score the answers?

No. In RLVR, the reward comes from a simple automatic checker, not from a human opinion. For math, it checks if the final number matches the correct answer, and for code, it checks if the code passes all unit tests. Because of this, the training loop can run millions of times without any human in the middle.

Can one model support both a thinking mode and a non-thinking mode?

Yes. Qwen3 is one unified model that supports both a thinking mode and a non-thinking mode, with an explicit thinking budget knob. DeepSeek's newer line, V3.1 and V4, also packs both thinking and non-thinking modes into one model.

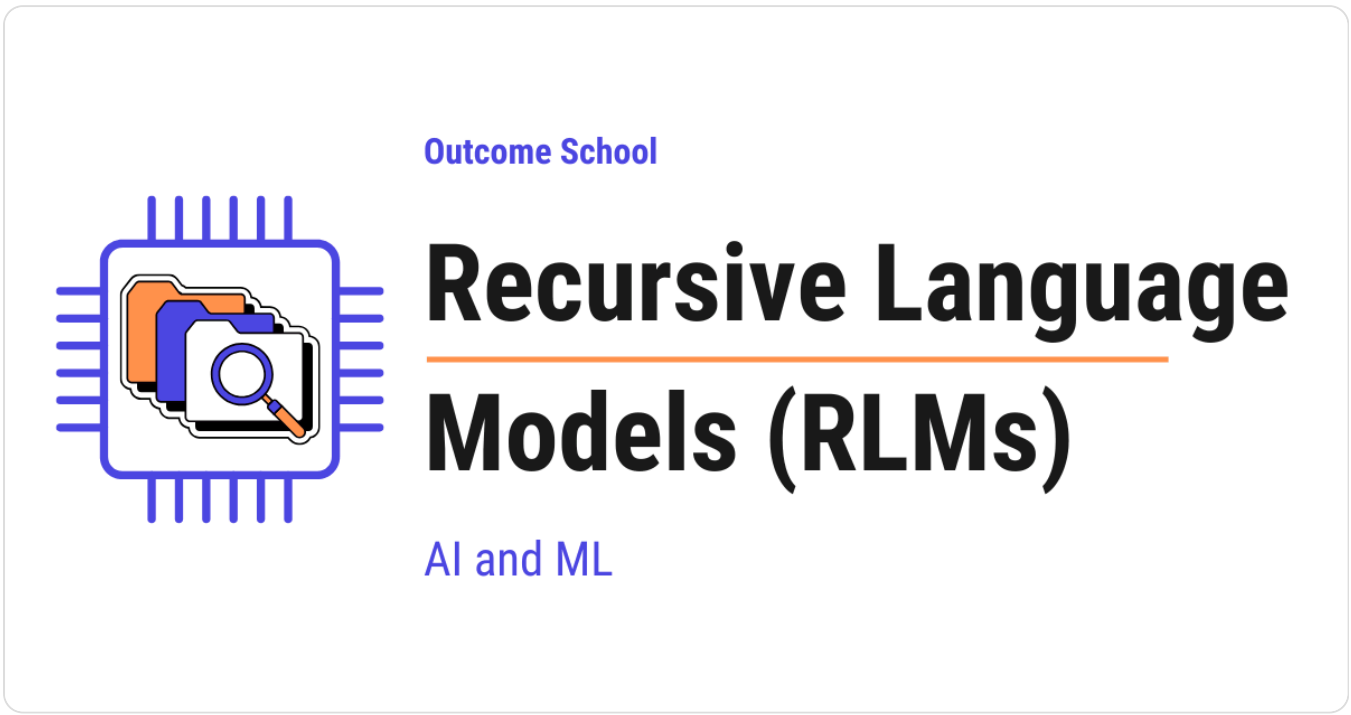
What is adaptive thinking in LRMs?

Adaptive thinking is when the model decides how much to think based on the complexity of the query. This is different from us setting a fixed thinking budget, like `1,024` or `32,768` tokens. Adaptive thinking helps, but for production traffic, a hard upper bound is the safer default.

LESSON 6.3

What are Recursive Language Models (RLMs) and How Do They Work?

Lesson 6.3 · ~12 min · Published May 19, 2026 · https://outcomeschool.com/blog/recursive-language-models



A Recursive Language Model (RLM) is a way of using a language model where the model can call another language model, or itself, on smaller parts of the input. This lets it handle very large inputs that do not fit in the context window.

In this blog, we will learn about **Recursive Language Models (RLMs)**, a new way of using language models to handle very large inputs that do not fit in the model's context window.

We will cover the following:

- What is a Recursive Language Model (RLM)?
- Why do we need RLMs?
- How an RLM works
- How the model writes and runs code
- Why RLMs work better
- Recursion inside RLMs
- How RLMs differ from simple chunking
- Advantages of RLMs
- Limitations of RLMs
- When to use RLMs

- RLM vs RAG
- A real use case

Let's get started.

What is a Recursive Language Model (RLM)?

A **Recursive Language Model (RLM)** is a way of using a language model where the model can call another language model (or itself) on smaller parts of the input to solve a bigger problem.

In simple words, instead of putting the entire input into the model at once, the model breaks the input into smaller pieces and calls another language model (or itself) on those smaller pieces.

RLM = Recursive + Language Model

Here, **Recursive** means the model can call itself (or another model) again and again on smaller parts of the problem. **Language Model** is the LLM that understands and generates text.

Why do we need RLMs?

The best way to learn this is by taking an example.

Let's say we have a very long document with 1 million tokens, and we want to ask a question about it.

When we put a lot of text into a language model, the following issues arise:

- The model has a context window limit. It cannot accept too many tokens at once.
- The quality of the answer goes down as the context gets longer. The model gets confused with too much information and misses important details in the middle. This is called **context rot**.
- The cost of running the model goes up.
- The speed of the model goes down.

This means, just making the context window bigger does not solve the problem. We need a better approach.

We have a detailed blog on The Lost in the Middle Problem in LLMs that covers this problem end to end.

So, here comes the Recursive Language Model to the rescue.

How an RLM works

In an RLM, the main model does not directly read the full input. Instead, the full input stays inside a Python environment as a variable. The model gets a small prompt that only names this variable and explains how to use it. The model then writes code to read parts of the input and call other language models on those parts. The smaller calls return short answers, which the main model combines into the final answer.

The next section walks through this with a concrete example.

Now, the question is: why is this called recursive? The answer is, the model can call another model, which can again call another model, and so on. Each call works on a smaller piece. This is just like recursion in programming.

How the model writes and runs code

Let's understand this with a real example. Suppose we have a long meeting transcript and we want to ask: "What were the action items?"

The model only generates text. It cannot run code by itself. So, we wrap the model with a Python environment. This environment has a variable `context` that stores the full transcript, and a function `call_llm(prompt)` that calls another language model.

Now, we send a short prompt to the main model:

```
Question: What were the action items?
Write Python code to answer this. The Python environment has:
- A variable named `context` that stores the full transcript.
- A function `call_llm(prompt)` that calls another language model.
```

Very important: The full transcript is never sent to the main model. Only the variable **name** `context` is sent in the prompt, not its actual content. The transcript stays inside the Python environment. The model can read parts of it through code, but the model never reads the whole thing. This is what keeps the main model's context small even when the input is huge.

Now, the loop runs as below:

Step 1: The model writes Python code as its response:

```
turns = context.split("\n")
results = [call_llm(f"Find action items in: {turn}") for turn in turns]
print(results)
```

Here, the model decides to split `context` by turn and call `call_llm` on each one.

Step 2: The code runs in the Python environment. Each `call_llm` call sends one turn to another language model and gets back a short answer. All answers are collected into a list.

Step 3: The list is sent back to the model. The model now sees only the short answers, not the full transcript.

Step 4: The model reads the list and returns the final list of action items as the answer. If the answer was not clear yet, the model would write more code and the loop would continue.

This back-and-forth between the model and the Python environment is called an **agentic loop**.

So, the model is only generating text. The text just happens to be Python code. The Python environment runs the code. And `call_llm` is just an API call to another language model.

Why RLMs work better

Here, we can notice that each smaller call gets a small input. In our transcript example, each `call_llm` call works on just one turn, not the full meeting. A small input is easy for the model to handle. The model can focus on the small piece and give a good answer.

Also, the main model only sees the small answers, not the full input. So, the main model has a clean and small context.

This way, we avoid context rot. The problem is solved.

Recursion inside RLMs

Now, let's understand the recursive part more clearly.

Let's say one piece is still too long for the smaller call. The smaller call can also act as an RLM. It can again split its piece into even smaller parts and call another language model on each part.

So, the structure looks like below:

- The main RLM works on the full input.
- The main RLM calls sub-RLMs on each big piece.
- Each sub-RLM can again call sub-RLMs on each smaller piece.
- The smallest calls work on tiny pieces of text.

This is just like a tree. The top of the tree handles the big problem. The leaves of the tree handle the small pieces. Each level passes its answer up to the next level.

Note: The depth of recursion depends on the size of the input and the type of question. The model itself decides how deep to go.

That's the beauty of Recursive Language Models.

How RLMs differ from simple chunking

Now, the next big question is: how is this different from just splitting the input into chunks and processing each chunk?

In **simple chunking**, we fix the chunk size beforehand. We send each chunk to the model, get an answer, and combine the answers. The split is fixed, and we always do the same steps.

In an **RLM**, the model itself decides how to split the input. The model also decides what to ask for each piece. The model can change its strategy based on the question.

So, how can we say RLM is smarter? The answer is, the RLM is more like a human reader. A human reader does not chunk a book into fixed pages. A human reader skims, jumps, reads carefully, and decides on the fly. An RLM works in a similar way.

This makes RLMs more flexible than simple chunking.

Advantages of RLMs

- **Handles very long inputs.** RLMs can work with inputs much larger than the model's context window.
- **Better accuracy on long context.** Since each call works on a small piece, the model can focus and give better answers.
- **Lower cost in some cases.** We do not need to send the full input every time. We only send small pieces when needed.
- **Modular thinking.** The model can break a big problem into small problems, just like a human would.
- **Flexible.** The model decides how to split the input based on the question.

Limitations of RLMs

- **More complex to build.** We need to set up a code environment and tool calls.
- **Many model calls.** One question can lead to many small calls. This can be slow.
- **Errors can stack up.** If a smaller call gives a wrong answer, the final answer can also become wrong.
- **Hard to debug.** Since the model writes its own code, it can be hard to know what went wrong.

So, RLMs are powerful, but we must use them carefully.

When to use RLMs

I will highly recommend using RLMs for the following cases:

- We have a very long document and need to ask questions about it.
- We need to analyze a large codebase.
- We need to summarize a long meeting or transcript.
- We need to compare many files at once.
- The task can be broken into smaller, independent sub-tasks.

For short inputs, we do not need RLMs. A simple language model call is enough. We must choose the approach based on our use case.

RLM vs RAG

Now that we have learned about RLMs, it's time to compare RLMs with another popular approach called RAG (Retrieval-Augmented Generation).

In **RAG**, we first search for the most relevant parts of a document and then send only those parts to the language model.

In **RLM**, the model itself decides how to explore the input. The model can write code, split the input, and call smaller models.

Let me tabulate the differences between RAG and RLM for your better understanding so that you can decide which one to use based on your use case.

Feature	RAG	RLM
How it picks data	A search step picks relevant parts first	The model itself explores the data
Flexibility	Less flexible, depends on the search step	More flexible, model decides what to do
Code execution	No code execution by the model	The model can write and run code
Best for	Question answering on a fixed knowledge base	Complex tasks on long inputs
Complexity	Easier to build	Harder to build

For simple question answering, RAG is enough. For complex analysis on long inputs, RLM is a better fit.

A real use case

A real use case in software development:

Let's say we want to find all the security issues in a large codebase. The codebase has hundreds of files. The total size is much bigger than what a normal model can read in one go.

With an RLM, we can do the following:

- Store the codebase as a variable.
- Ask the RLM: "Find all security issues."
- The RLM writes code to loop through each file.
- For each file, the RLM calls a smaller language model to find security issues in that file.
- The smaller calls return short lists of issues.
- The RLM combines all the issues into a final report.

Here, we can see that the main RLM does not read the full codebase. The main RLM only sees small summaries from each file. This keeps the main context clean and small.

This way we can use RLMs to solve the interesting problem of large codebase analysis.

Summary

We have learned the following:

- A Recursive Language Model (RLM) is a way of using a language model that can recursively call other language models on smaller parts of the input.
- RLMs solve the problem of long context and context rot.
- The main model writes code to split the input and call smaller models.
- RLMs are powerful for long documents, codebases, and complex tasks.
- We must use RLMs carefully because they have more complexity and more model calls.

Now, we have understood Recursive Language Models (RLMs).

This way we can use RLMs to solve any problem on long inputs in a very simple way.

Frequently Asked Questions

Does a bigger context window solve the long input problem?

No. Just making the context window bigger does not solve the problem. As the context gets longer, the quality of the answer goes down because of context rot, the cost goes up, and the speed goes down.

An RLM avoids this by keeping each call's input small.

Does the main model in an RLM ever read the full input?

No. The full input stays inside a Python environment as a variable, and only the variable name is sent in the prompt. The main model reads parts of the input through code and sees only the short answers from the smaller calls. This keeps its context clean and small even when the input is huge.

Can the language model run the Python code by itself?

No. The model only generates text, and that text just happens to be Python code. The Python environment wrapped around the model runs the code. The `call_llm` function inside that code is simply an API call to another language model.

Who decides how deep the recursion goes in an RLM?

The model itself decides how deep to go. The depth depends on the size of the input and the type of question. If one piece is still too long, the smaller call can act as an RLM and split its piece again into even smaller parts.

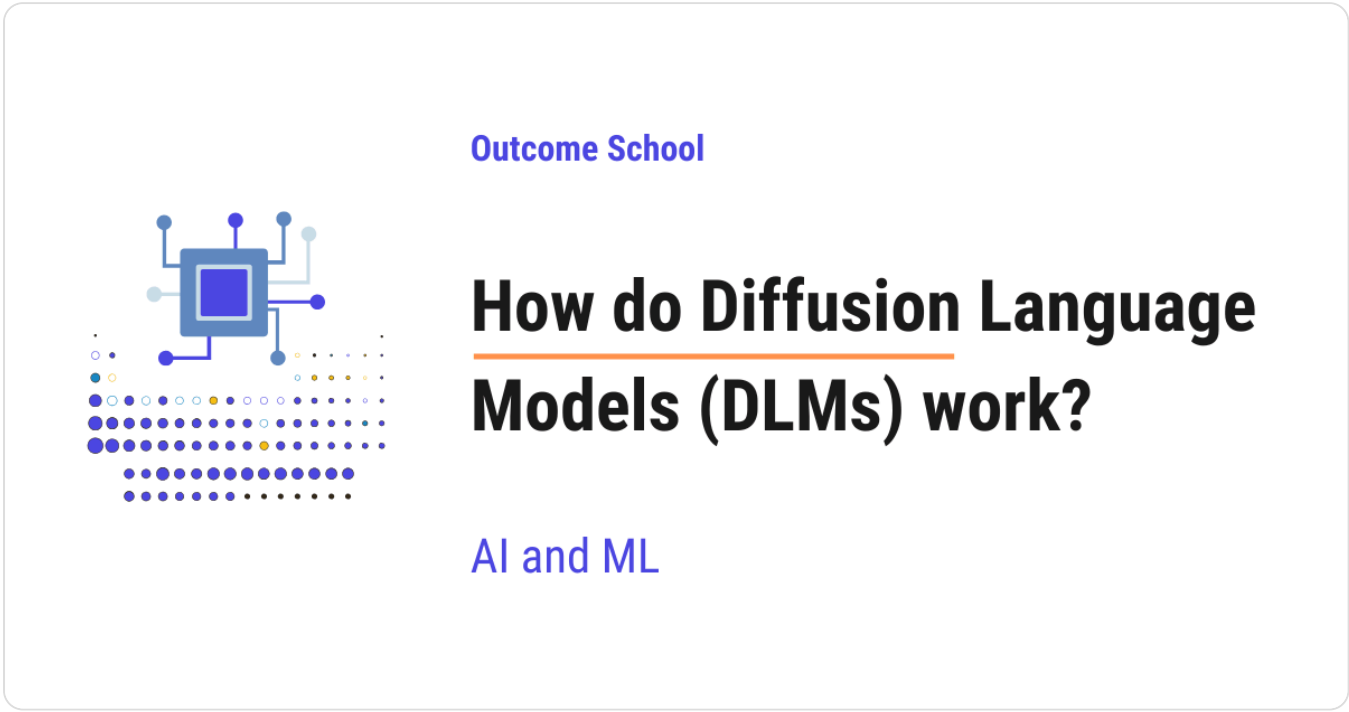
Do we need an RLM for short inputs?

No. For short inputs, a simple language model call is enough. RLMs make sense for very long documents, large codebases, long transcripts, comparing many files, or tasks that can be broken into smaller independent sub-tasks.

LESSON 6.4

How do Diffusion Language Models (DLMs) work?

Lesson 6.4 · ~17 min · Published June 22, 2026 · https://outcomeschool.com/blog/how-do-diffusion-language-models-dlms-work



A Diffusion Language Model (DLM) is a type of AI model that writes text by starting from pure gibberish and slowly cleaning it up into a clear sentence. It does not write one word at a time from left to right like the usual LLMs.

In this blog, we will learn about Diffusion Language Models (DLMs), a new way to make models write text. They promise to generate words in a different way than the LLMs we use today, and that too much faster in many cases.

We will cover the following:

- What is a Diffusion Language Model?
- How do today's language models write text?
- The problem with the usual approach
- Where the diffusion idea comes from
- What does "noise" mean for text?
- The two phases: forward and reverse
- How a DLM actually generates text, step by step
- A tiny end-to-end example
- A simple code-style walk-through

- DLMs vs the usual language models
- Advantages of DLMs
- Limitations of DLMs
- The current state

Let's get started.

What is a Diffusion Language Model?

A Diffusion Language Model is a type of AI model that writes text by starting from a piece of pure gibberish and slowly cleaning it up into a clear, meaningful sentence.

In simple words, it does not write a sentence one word at a time from left to right. Instead, it starts with a rough, messy version of the whole sentence and then refines it again and again until it becomes correct.

Let's understand the term: Diffusion

Diffusion is a word borrowed from nature. It means something spreading out and mixing, like a drop of ink slowly spreading into a glass of water until everything is blurry.

So, a Diffusion Language Model is a model that learns to take blurry, mixed-up text and turn it back into clean text.

Do not worry, we will learn about each part in detail.

How do today's language models write text?

Before jumping into diffusion, we must understand how the popular LLMs today write text. This will help us see why diffusion is interesting.

LLMs are built on an architecture called the **Transformer**, which is just a powerful kind of AI brain that is very good at understanding language. These models write in a style called **autoregressive** generation.

Let's decompose that word:

> Autoregressive = Auto + Regressive

In simple words, it means the model predicts the next word based on all the words that came before it. It writes one word, looks at everything written so far, then writes the next word, and keeps going.

Let's say we ask the model to complete this sentence: "The sky is".

The model writes one word at a time, like below:

```
Step 1:  The sky is ___             ->  picks "blue"
Step 2:  The sky is blue ___        ->  picks "and"
Step 3:  The sky is blue and ___    ->  picks "clear"
```

Here, we can see that the model adds one word in each step. It can never pick the third word before it has picked the second word. Each word waits for the one before it.

This is how today's language models write text. It works very well, and that is why it feel so natural.

The problem with the usual approach

Now, the question is: if this works so well, why do we need anything new?

The issue with this approach is **speed**. Because the model writes strictly one word after another, it cannot do them at the same time. If a sentence has 100 words, the model must take 100 turns, one after another. It cannot jump ahead.

Let's say we are writing a long essay. Writing one word, waiting, then the next word, waiting, then the next, is slow. We are stuck in a single line, moving forward one step at a time. We cannot go back and fix an earlier word either, because we have already moved past it.

Now, the next big question is: what if a model could work on the whole sentence at once, fixing many words together instead of one at a time?

So, here comes the Diffusion Language Model to the rescue.

Where the diffusion idea comes from

Before jumping into text, we must know where this idea was born.

The diffusion idea first became famous for creating images. Tools that turn a text prompt into a beautiful picture often use a method called **diffusion**.

Here is the simple idea behind image diffusion. We take a clear photo and slowly add random dots of noise to it, again and again, until the photo becomes complete static, like an old TV with no signal. Then we train a model to do the reverse: start from pure static and slowly remove the noise until a clear picture appears.

We can picture it like below:

```
Clear image  ->  add noise  ->  more noise  ->  pure static
                                                     |
                                                     v  (now reverse it)
Clear image  <-  remove noise  <-  less noise  <-  pure static
```

Here, we can see that the model learns to walk backward, from total mess to a clean image, one small cleanup step at a time.

Now, the next big question is: can we do the same thing for text instead of images? The answer is yes, and that is exactly what a Diffusion Language Model does.

What does "noise" mean for text?

For an image, adding noise was easy. We just add random dots to the pixels.

But text is different. A sentence is made of words, not pixels. We cannot add a "half word". So we need a different idea of noise for text.

For text, **noise means hiding or replacing words with a blank or a random word.**

In simple words, to make a clean sentence noisy, we hide some of its words. The more words we hide, the noisier the sentence becomes. When every word is hidden, the sentence is pure noise. This is just like the ink drop spreading in water until everything is blurry.

Let's say our clean sentence is "The cat sat on the mat". We use the symbol `[?]` for a hidden word. Adding more and more noise looks like below:

```
Clean:        The cat sat on the mat
A little noise: The cat [?] on the mat
More noise:     The [?] [?] on [?] mat
Pure noise:     [?] [?] [?] [?] [?] [?]
```

Here, we can see that as we add more noise, more words turn into `[?]`. At the end, the whole sentence is just blanks. This blank-filled sentence is the text version of "pure static".

This way of hiding words is called **masking**, and the hidden spot is called a **mask**. So, for text, adding noise simply means masking more words.

This is how we bring the idea of noise into the world of text. It makes our life easy, because hiding words is something a computer can do instantly.

The two phases: forward and reverse

A Diffusion Language Model works in two phases. Let's understand each one.

The forward phase (adding noise): This is the easy part, and it happens only during training. We take a clean sentence from our training data and slowly hide its words until the whole sentence is masked. We do not need any intelligence for this. We are just deleting words on purpose.

The reverse phase (removing noise): This is the hard part, and this is what the model actually learns. The model takes a masked sentence and tries to guess the hidden words to make it clean again. It does this again and again, filling in more words each time.

We can picture both phases like below:

```
FORWARD (training only, just hide words)
The cat sat on the mat  ->  The [?] sat on [?] mat  ->  [?] [?] [?] [?] [?] [?]
REVERSE (what the model learns to do)
[?] [?] [?] [?] [?] [?]  ->  The [?] sat on [?] mat  ->  The cat sat on the mat
```

Here, we can see that the forward phase destroys the sentence and the reverse phase rebuilds it. The model only needs to become very good at the reverse phase. The forward phase is just a teacher creating practice questions.

So, during training, we hide words and ask the model to guess them. By practicing on millions of sentences, the model gets very good at filling in blanks. This is how it learns. It is just like a student who keeps solving fill-in-the-blanks exercises until the answers come naturally.

How a DLM actually generates text, step by step

Now comes the most interesting part. Till now, we have learned how the model is trained. Now it's time to learn how it writes a brand new sentence for us.

It starts with a sentence that is all blanks and refines it step by step. Let's understand the full process.

Step 1: Start from pure noise. The model begins with a sentence where every word is masked. It is a blank canvas of `[?]` symbols.

Step 2: Guess all the words at once. Here is the beauty of it. Unlike the usual models, the DLM looks at every blank at the same time and makes a guess for each one in a single pass. It does not go left to right. It works on the whole sentence together.

Step 3: Keep the confident words, blank out the rest. Some guesses will be good and some will be shaky. The model keeps the words it is most confident about and turns the unsure ones back into `[?]`. This is the key trick that makes the result better over time. Do not worry, we will see this with a clear example soon.

Step 4: Repeat. The model looks at the partly-filled sentence and guesses the remaining blanks again, now with more context to help it. It keeps the confident ones and blanks the rest.

Step 5: Stop when nothing is masked. After a few rounds, there are no blanks left. The sentence is complete.

We can picture the whole generation like below:

```
Round 1:  [?]  [?]  [?]  [?]  [?]      (start, all masked)
            |    |    |    |    |
            v    v    v    v    v       (guess all at once, keep the confident ones)
Round 2:  The  [?]  [?]   is  [?]
            |    |    |    |    |
            v    v    v    v    v       (guess the rest again)
Round 3:  The  food [?]   is good
            |    |    |    |    |
            v    v    v    v    v
Round 4:  The  food here  is good       (done, no blanks left)
```

Here, we can see that the model fills in the easy, confident words first, like "The" and "is". Then it uses those words as hints to fill in the harder words like "food", "here", and "good". Step by step, the messy blank sentence becomes a clean, meaningful one.

This is how a Diffusion Language Model generates text. The big difference is that it can fix many words in parallel, instead of one at a time. It enables us to do complex things very simply.

A tiny end-to-end example

The best way to learn this is by taking an example. Let's say we ask the model to answer: "What is the capital of France?"

The model needs to produce a short answer of, let's say, 4 word-slots. It starts with all of them masked.

```
Question: What is the capital of France?
Answer slots:  [?]  [?]  [?]  [?]
```

Round 1: The model guesses every slot at once. Suppose it produces:

```
The   is   capital   Paris
```

The model is very confident about "Paris" and fairly confident about "The". It is unsure about "is" and "capital" in those spots. So it keeps the confident words and blanks the rest.

```
The   [?]   [?]   Paris
```

Round 2: Now the model looks at "The [?] [?] Paris" and guesses the two blanks again. With "The" and "Paris" already in place, the context is clearer. This time it produces:

```
The   capital   is   Paris
```

Here, we can see that the answer is now correct and complete. The model used the words it was sure about as anchors to figure out the rest.

So, in just two rounds, the model went from total blanks to a correct answer. This way we can use a Diffusion Language Model to write text by refining the whole sentence together.

A simple code-style walk-through

Now, let's see this idea in a small, simplified code-style form using Python. This is just for the sake of understanding, not a real library. We can write it as below:

```
# A simplified view of how a DLM generates text
sentence = ["[?]", "[?]", "[?]", "[?]"]   # start: everything masked
for current_round in range(num_rounds):
    # 1. Ask the model to guess EVERY position at once
    guesses, confidence = model.predict_all(sentence)
    # 2. Keep only the most confident guesses, mask the rest
    for i in range(len(sentence)):
        if confidence[i] > threshold:
            sentence[i] = guesses[i]      # lock in this word
        else:
            sentence[i] = "[?]"           # send it back to be re-guessed
    # 3. Stop early if no masks are left
    if "[?]" not in sentence:
        break
print(sentence)   # the final clean sentence
```

Here, we have written the core loop of a Diffusion Language Model. Let me explain each part.

- `sentence` starts as a list full of `[?]`, which means every position is masked.
- `model.predict_all(sentence)` is the key step. The model guesses all positions at the same time and also tells us how confident it is about each guess.
- The `if confidence[i] > threshold` check keeps the words the model is sure about and sends the unsure ones back to `[?]` to be guessed again next round. Here, `threshold` is just a confidence bar we set, for example "only keep words the model is more than 90 percent sure about".
- The loop repeats, and each round the sentence gets cleaner.
- When there are no `[?]` left, we stop, and the sentence is ready.

Notice that there is no left-to-right loop here. The model works on the whole sentence in every round. This is the heart of the difference. Here, we can see that many words can be decided together.

DLMs vs the usual language models

Now that we have learned how a DLM works, it's time to compare it with the usual autoregressive models we discussed earlier.

The usual model is a decoder-only Transformer that writes left to right, one word at a time. The diffusion model fills the whole sentence and refines it over a few rounds. We can picture the contrast like below:

```
Autoregressive (usual):                Diffusion (DLM):
word1                                   [?] [?] [?] [?] [?]   (all at once)
word1 word2                                   |
word1 word2 word3                             v
word1 word2 word3 word4                 the [?] cat on mat   (refine)
... one new word each step                    |
                                              v
                                        the big cat on mat   (refine again)
one direction, one word per step        whole sentence, refined over rounds
```

Here, we can see the core difference. The usual model takes as many steps as there are words. The diffusion model takes a small number of refinement rounds that does not grow with every extra word, and it works on many words together in each round.

Let me tabulate the differences between Autoregressive Models and Diffusion Language Models for your better understanding.

Feature	Autoregressive Model	Diffusion Language Model
Writing order	Left to right, one word at a time	Whole sentence at once, refined in rounds
Parallel work	No, each word waits for the previous	Yes, many words decided together
Number of steps	Equal to the number of words	A small number of rounds, not tied to the word count
Can revise earlier words	No, once written it stays	Yes, it can re-mask and fix words

One important thing to clear up: Diffusion Language Models (DLMs) often use a Transformer-based architecture internally, similar to the large language models. The difference is not the brain, it is the way the text is written. The usual model writes one word at a time, and the diffusion model fills and refines the whole sentence together.

Advantages of DLMs

- **Speed through parallel work:** Because the model decides many words together, it can produce text in far fewer steps than writing one word at a time. This can make generation much faster.
- **It can fix its own mistakes:** A word that was placed early can be sent back to `[?]` and corrected in a later round. The usual model cannot undo a word once it is written.
- **Whole-sentence view:** The model sees the entire sentence in every round, so a word near the end can influence a word near the start. This global view can help with consistency.
- **Good control over the output:** Since the model fills in blanks, it is naturally suited to tasks where we give a partial sentence and ask it to complete the missing parts.

Limitations of DLMs

- **Choosing how many rounds is tricky:** Too few rounds give messy text, and too many rounds slow things down. Finding the right balance takes care.

- **Fixed length can be awkward:** Many DLM setups need to know how many word-slots to fill ahead of time. The usual models can simply keep writing until they decide to stop, which feels more natural for open-ended writing.
- **Less mature tooling:** The usual autoregressive models have years of tuning, tools, and tricks behind them. DLMs are catching up but are still younger.
- **Quality is still being proven:** For some tasks, the very best autoregressive models are still ahead on quality, though the gap keeps shrinking.

So, DLMs are a promising direction, but they are still an active area of research.

The current state

Now, let's discuss where things stand today, in 2026.

Diffusion for text started as a research idea, but it has moved quickly into real, working models. Researchers have built large diffusion-based language models that can chat, answer questions, and write code, and some of them run noticeably faster than the usual word-by-word models because of the parallel refinement.

A few important points about the current state:

- Several real diffusion-style language models are now in use, and the main selling point is high speed. Their speed is measured in **tokens** per second, where a token is a small chunk of a word. For example, **Mercury** from Inception Labs can generate over a thousand tokens per second on a single chip, which is several times faster than the usual word-by-word models. Google has released **DiffusionGemma**, an open model that anyone can download and run, and it reaches a similar speed on a single chip. **LLaDA** is another popular open model that anyone can download and run.
- Newer diffusion models can also **reason**. They no longer just write fast, they can solve hard math and coding problems too, which earlier diffusion models struggled with.
- Many practical systems are **hybrids**. They borrow the strong Transformer brain from the usual models and add the diffusion-style refinement on top, so they get the best of both worlds. A common trick is to start from a normal trained model and then continue training it with the diffusion idea, instead of starting from scratch.

So, now we know where Diffusion Language Models fit and how they work.

Frequently Asked Questions

Do Diffusion Language Models use a Transformer?

Often, yes. Diffusion Language Models often use a Transformer-based architecture internally, similar to the usual large language models. The difference is not the brain, it is the way the text is written. The

usual model writes one word at a time, while the diffusion model fills and refines the whole sentence together.

Are Diffusion Language Models better than autoregressive LLMs?

Not in every way yet. DLMs can be much faster because they decide many words together, and they can fix their own mistakes. But for some tasks, the very best autoregressive models are still ahead on quality, though the gap keeps shrinking. DLMs also have less mature tooling and are still an active area of research.

Can a Diffusion Language Model change a word it already wrote?

Yes. A word placed in an early round can be sent back to a mask and corrected in a later round. The model keeps only the words it is confident about and re-guesses the rest. The usual autoregressive model cannot undo a word once it is written.

Does a Diffusion Language Model need to know the output length in advance?

Often, yes. Many DLM setups need to know how many word-slots to fill ahead of time. The usual autoregressive models can simply keep writing until they decide to stop, which feels more natural for open-ended writing.

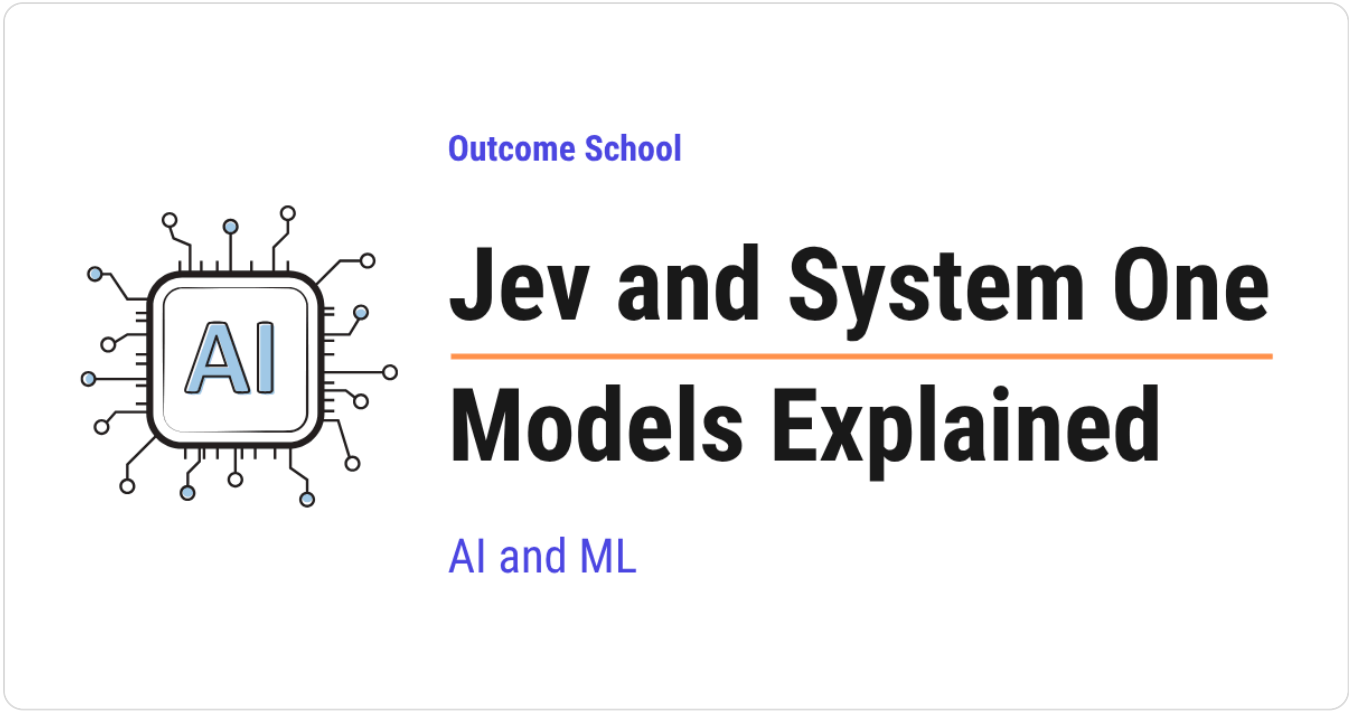
Do we have to train a Diffusion Language Model from scratch?

No. A common trick is to start from a normal trained model and then continue training it with the diffusion idea, instead of starting from scratch. Many practical systems are hybrids that borrow the strong Transformer brain from the usual models and add diffusion-style refinement on top.

LESSON 6.5

Jev and System One Models Explained

Lesson 6.5 · ~18 min · Published September 17, 2026 · https://outcomeschool.com/blog/jev-and-system-one-models-explained



Jev is the first System One Model, a new kind of AI model that does not write text at all. It looks at a situation and returns a decision from a fixed list of answers, along with how sure it is, that our software can use directly.

In this blog, we will learn about Jev and System One Models, a new kind of AI model that does not write text at all but only makes fast decisions that our software can use directly. We will also see what System One and System Two thinking mean, why using a normal LLM for small decisions is slow and unreliable, how Jev answers many questions in a single pass instead of one word at a time, what a typed answer is, how the probability attached to every answer is made honest, why Jev cannot hallucinate, and where it works well and where it fails.

We will cover the following:

- What is a System One Model?
- System One vs System Two Thinking
- The Problem with Using an LLM for Decisions
- What is Jev?
- How Jev Works
- Typed Answers: Choice, Score, and Yes/No
- Calibration and RLCD
- Why Jev Cannot Hallucinate

- Jev vs LLM
- Where Jev Works Well and Where It Fails
- When to Use Which One

Let's get started.

What is a System One Model?

A System One Model is an AI model that looks at a situation and returns a decision, not text.

In simple words, we give it some information and a fixed list of possible answers. It picks one answer from that list and also gives a number telling how sure it is.

Let's say we run an online shop. A customer sends a message: "My card was charged twice." We want to know which team must handle this message. The possible answers are `billing`, `technical`, and `account`. A System One Model reads the message and returns `billing` with a probability of 0.97, which means it is 97% sure.

That is all. It does not write a reply to the customer. It does not explain its thinking. It just decides.

System One Model = System One + Model

Here, "System One" comes from a famous idea about how the human brain thinks. And "Model" is the AI model. So, a System One Model is an AI model that thinks like the System One of our brain.

Before jumping into Jev, we must know what System One and System Two thinking mean.

System One vs System Two Thinking

Daniel Kahneman, a psychologist who won the Nobel Prize, explained that our brain has two ways of thinking.

System 1 is fast, automatic, and effortless. When we see a friend's face, we know who it is instantly. When someone asks "2 + 2?", we say "4" without thinking. We do not sit and calculate. The answer just comes.

System 2 is slow, careful, and effortful. When someone asks "17 × 24?", we stop, think step by step, and work it out. It takes time and energy.

Both are useful. System 1 handles thousands of small decisions every day. System 2 handles the hard ones.

Note: Kahneman writes them as System 1 and System 2. TypeSafe writes the model name as System One. Both mean the same thing.

Now, let's connect this to AI.

An LLM, which stands for Large Language Model, is the AI model behind a chatbot like ChatGPT. It is built to produce text, one small piece at a time. Even when we ask it a simple yes or no question, it goes through this slow writing process. And the newer reasoning models go one step further, they write out their thinking step by step before giving the final answer. This is System 2 style work. It is powerful, but slow and costly.

But most of the decisions inside real software are small and repeated. Is this email spam or not? Which team must get this ticket? Is this review positive or negative? Is this answer safe to show to the user? These are System 1 type decisions. We need an instant answer, and we need it millions of times a day.

So, the question is: why are we using a slow, System 2 style model for fast, System 1 style decisions?

This is exactly the problem that System One Models are built to solve.

The Problem with Using an LLM for Decisions

The best way to learn this is by taking an example.

Suppose we have a support system and we want to send every incoming ticket to the right team. Today, most of us do it with an LLM as below:

```
prompt = """
Classify this support ticket into one of: billing, technical, account.
Reply with only the category name.
Ticket: My card was charged twice.
"""
response = llm.generate(prompt)
category = response.text.strip()
```

Here, we are asking the LLM to read the ticket and reply with only the category name. It looks simple. But, here is the catch.

Problem 1: It writes the answer one token at a time.

A token is a small piece of text, roughly a word or a part of a word. An LLM writes its answer one token at a time. To write "billing", it first predicts "bill", then "ing", and so on. Every single step is a full pass through a huge model. So, even a one-word answer takes many steps. And if the model decides to add an explanation, it takes even more steps.

Problem 2: We cannot trust the output format.

We asked for only the category name. But the model can reply "The category is billing." or "Billing." or "billing, let me know if you need anything else." or even "refund", which is not in our list at all. Now we must write extra code to clean this output. Most of the time, we do mistakes while writing this cleaning code, and it breaks in surprising ways.

Problem 3: We do not know how sure it is.

The model says "billing". Was it 99% sure or 51% sure? We do not know. So, we cannot decide when to trust it and when to send the ticket to a human.

Problem 4: It is slow and costly.

A frontier LLM, which means one of the biggest and most capable models available today, takes seconds for such a call. If we have a million tickets a day, both the waiting time and the bill add up very fast.

We needed a model that only decides, does it instantly, always returns a valid answer, and tells us how sure it is.

So, here comes Jev to the rescue.

What is Jev?

Jev is the first System One Model, built by a company called TypeSafe AI.

It was announced on September 15, 2026. TypeSafe AI was founded by Diogo Almeida, one of the people behind ChatGPT, and the team spent about two years quietly building a new way of training models. Jev is the result of that work.

The most surprising thing about Jev is this: **it does not generate text at all.**

We cannot chat with Jev. We cannot ask it to write an email or code. It takes a situation as input, and it returns typed decisions with probabilities as output. Nothing else.

Now, let's understand how it works.

How Jev Works

Jev takes two things as input:

- **State**: the information about the situation. State is just a word for "everything the model needs to know about the situation right now". It is always text. It can be a plain string, a list of texts, or a JSON object, which is a simple way of writing data as labels and values. For example, the customer message, their last few transactions, and the refund policy.
- **Questions with allowed answers**: what we want to decide, and what the valid answers are. For example, "Which team?" with the allowed answers `billing`, `technical`, and `account`.

And it returns:

- **Typed answers**: one answer per question, always chosen from the allowed answers.
- **Probabilities**: for every answer, a number between 0 and 1 telling how sure the model is.

Let's see it in code. For the sake of understanding, the code below is simplified and is not the exact API:

```
result = jev.decide(
    state={
        "message": "My card was charged twice.",
        "recent_transactions": ["$49 on Sep 10", "$49 on Sep 10"]
    },
    questions={
        "team": ["billing", "technical", "account"],
        "is_urgent": "yes_no",
        "frustration": {"min": 0, "max": 2}
    }
)
```

The result will be:

```
{
  "team": { "answer": "billing", "probability": 0.97 },
  "is_urgent": { "answer": true, "probability": 0.88 },
  "frustration": { "answer": 1, "probability": 0.74 }
}
```

Here, we can notice a few things:

- We asked three questions in one call and got three answers.
- Every answer is one of the allowed answers. `team` can only be `billing`, `technical`, or `account`. It can never be "refund" or a sentence.
- Every answer comes with a probability.

Now, the most important part: **Jev answers all these questions in a single parallel pass.**

Here, parallel means all at the same time, and a pass means one run through the model. Let's understand what this means with an analogy.

An LLM is like a person writing an essay with a pen. Each word comes after the previous word. It cannot write the tenth word before writing the ninth.

Jev is like a person filling a form with checkboxes. It reads the whole state once, and then ticks all the boxes at the same time. There is no "next word" to wait for. That's the beauty of a System One Model.

This is why Jev is so fast. A decision that takes a frontier LLM 3 seconds to a few minutes takes Jev about 70 to 500 milliseconds, which is a fraction of a second. TypeSafe reports that Jev is 40 to 200 times faster than LLMs of comparable intelligence on these tasks. In their own evaluation, they report Jev to be up to 193 times faster than Claude Sonnet 5 on decision tasks.

And because there is no text output, there is nothing to charge for on the output side. Jev costs $0.042 per million input tokens and the output is free.

So, it decides instantly, always returns a valid answer, tells us how sure it is, and costs very less. All four problems that we saw with the LLM approach got solved.

This was all about how Jev works. Now, let's understand what "typed" means, because this is the heart of Jev.

Typed Answers: Choice, Score, and Yes/No

In programming, a **type** tells what kind of value something can hold. A variable of type boolean can only hold `true` or `false`. It can never hold "maybe". This is what we call **type safety**. The program cannot even represent a wrong kind of value. This is also where the company name TypeSafe comes from.

Jev brings the same idea to AI answers. Every question we ask has a type, and the answer must fit that type. Jev supports three kinds of typed answers:

Choice: pick one option from a fixed list. Example: `billing`, `technical`, or `account`. Jev supports up to 255 options in one list.

Score: pick a number from a fixed range. Example: frustration level from 0 to 2, or a quality score from 1 to 5.

Yes/No: a `true` or `false` answer. Example: "Is this urgent?"

And every one of these comes with a probability.

Now, the question is: how does Jev make sure the answer always fits the type?

Answer: Jev does not write the answer as text, it scores the options. Just for the sake of understanding, think of it like this. For a choice question with three options, Jev produces three probabilities, one for each option, and they add up to 1. The answer is simply the option with the highest probability. There is no way to output a fourth option, because there is no fourth slot.

This is very different from asking an LLM to "reply in JSON". With an LLM, the JSON is still written token by token, and we hope it comes out valid. With Jev, the shape of the answer is fixed before the model even runs.

Calibration and RLCD

Till now, we have learned that Jev returns a probability with every answer. Now, the next big question is: can we trust that probability?

This is where **calibration** comes into the picture.

A model is calibrated when its probabilities match reality. In simple words, if we take all the answers where the model said "I am 90% sure", then about 90% of them must actually be correct. Not 60%. Not 99%. Around 90%.

Let's say a weather app says "70% chance of rain" on 100 different days. If it rained on about 70 of those days, the app is calibrated. If it rained on only 30 of those days, the app is not calibrated, and its 70% means nothing.

Most LLMs are not trained for this. They are trained to write text that humans like. So, when an LLM says "I am confident", it is just writing words that sound confident.

Jev is trained differently, using a method that TypeSafe calls **RLCD**.

RLCD = Reinforcement Learning for Calibrated Decisions

Let's break it down.

Reinforcement Learning is a way of training where the model makes a decision, gets a reward when the decision turns out to be good, and gets a penalty when it turns out to be bad. Over millions of tries, the model learns to make better decisions.

For Calibrated Decisions means the reward is not just for being right. The reward is for being right and being honest about how sure it is. Let's see with an example:

- The model says 99% and is right: big reward.
- The model says 99% and is wrong: big penalty.
- The model says 55% and is right: small reward.
- The model says 55% and is wrong: small penalty.

So, the only way for the model to score well is to say a high probability when it is truly likely to be right, and a low probability when it is not. This is how the probabilities become honest.

And this honest probability is very useful for us. Now, we can write code like below:

```
if result["team"]["probability"] > 0.9:
    route_to(result["team"]["answer"])
else:
    send_to_human()
```

Here, we are trusting Jev when it is very sure, and sending the ticket to a human when it is not. We could never write these lines with a normal LLM, because it never gave us an honest number. This makes our life easy.

Note: Calibration does not mean every single answer is correct. A 90% answer is still wrong 10% of the time. Calibration only means the number is honest, so we know how much to trust it.

Why Jev Cannot Hallucinate

You must have heard that LLMs hallucinate. In simple words, hallucination means the model confidently says something that is not true or does not exist. It can invent a book that was never written, a function that does not exist, or a category that is not in our list.

TypeSafe makes a strong claim: Jev cannot hallucinate. Let's understand exactly what this means, because it is easy to misread.

There are two kinds of mistakes a model can make when we ask it to decide something:

Type error: the answer is not even a valid kind of answer. We asked for `billing`, `technical`, or `account`, and the model said "refund" or wrote a paragraph.

Wrong answer: the answer is valid, but it is the wrong one. The ticket was about a login problem, and the model said `billing`.

Jev can never make a type error. This is a mathematical guarantee, not a promise. The answer is always one of the allowed options, because the model only scores the allowed options. There is no path inside the model that can produce anything else. TypeSafe even says that a single counterexample would be enough to prove this claim false.

Jev can still give a wrong answer. But when it is unsure, the probability will be lower, because it is calibrated. So, most of the wrong answers can be caught by looking at the probability, and we can send those cases to a human.

This is the meaning of "cannot hallucinate" here: **it can never invent an answer that does not exist, and when it is unsure, it tells us.**

One more thing to notice. Normally, we think System 1 is fast but error-prone, and System 2 is slow but reliable. TypeSafe turns this idea around. For these decision tasks, a System One Model can be made more reliable than the slow alternative, because its answers are limited to the allowed options by design, and its confidence is honest by training.

Jev vs LLM

Now that we have learned about Jev in detail, it's time to compare it with an LLM side by side.

Let me tabulate the differences between Jev and an LLM for your better understanding.

Aspect	Jev (System One Model)	LLM
Output	Typed decisions with probabilities	Free-form text
How it answers	All answers in one parallel pass	One token at a time
Can write text, code, replies	No	Yes
Can reason step by step	No	Yes
Output format guarantee	Always valid, by design	Must be cleaned and checked by our code
Confidence	Calibrated probability with every answer	Not available or not honest
Speed	70 to 500 milliseconds	Seconds to minutes
Cost	$0.042 per million input tokens, output free	Much higher, and output tokens cost extra
Input	Text, JSON, lists of text	Text, images, audio, and more
Best for	Classify, route, score, check, decide	Chat, write, code, reason, explain

---

Where Jev Works Well and Where It Fails

Where it works well:

- **Routing**: which team, which model, or which workflow must handle this?
- **Classification**: spam or not, positive or negative, which category?
- **Scoring**: how relevant is this document, or how good is this answer, from 1 to 5?
- **Guardrails**, which means safety checks: is this LLM output safe, on topic, and correct? Jev can check an LLM's answer in a fraction of a second before the user sees it.
- **Real-time decisions**: anything inside a user-facing app where we cannot wait for seconds.
- **Large datasets**: running the same decision over millions of rows, where cost and speed matter the most.

Where it fails:

- **Writing anything**: Jev cannot write a reply, an email, a summary, or code. There is no text output.
- **Open-ended reasoning**: if the task needs many steps of thinking, like solving a hard math problem, Jev is not the right tool.
- **Explanations**: Jev gives the answer and the probability, not the reason behind it.
- **Images and audio**: as of now, Jev accepts text only.
- **Very large option lists**: one choice question supports up to 255 options.

When to Use Which One

The simplest rule is this:

If the answer is a decision, use Jev. If the answer is text, use an LLM.

And in real systems, we use both together. Let's say we are building a customer support system that replies to customers automatically.

First, Jev reads the ticket and decides the team, the urgency, and whether it is safe to reply automatically. This takes a fraction of a second.

Then, if Jev is sure it is safe, the LLM writes the reply.

After that, Jev checks the reply and decides whether it is correct and polite before it is sent.

Here, the LLM does the System 2 work: writing and reasoning. Jev does the System 1 work: deciding, fast and many times. Each one does what it is good at.

This kind of flow, where one step decides the path and another step does the work, is a common pattern. We have a detailed blog on AI Orchestration that covers these patterns end to end.

Now we must have understood Jev and System One Models, how they think like the fast System 1 of our brain, how they answer all questions in a single pass instead of one token at a time, how typed answers make wrong formats impossible, how RLCD makes the probability honest, and when to use them along with an LLM.

Frequently Asked Questions

Who built Jev?

Jev was built by a company called TypeSafe AI and announced on September 15, 2026. TypeSafe AI was founded by Diogo Almeida, one of the people behind ChatGPT. The team spent about two years building a new way of training models, and Jev is the result of that work.

Can we ask Jev several questions in one call?

Yes. We can ask many questions in one call, and Jev answers all of them in a single parallel pass. It reads the whole state once and then ticks all the answers at the same time, like filling a form with checkboxes. Each answer comes with its own probability.

Is asking an LLM to reply in JSON the same as Jev's typed answers?

No. With an LLM, the JSON is still written token by token, and we hope it comes out valid. With Jev, the shape of the answer is fixed before the model even runs. Jev scores only the allowed options, so it

cannot output anything outside them.

Does a calibrated probability mean Jev is always right?

No. Calibration does not mean every single answer is correct. A 90% answer is still wrong about 10% of the time. Calibration only means the number is honest, so we know how much to trust it and can send the unsure cases to a human.

How much does Jev cost to use?

Jev costs $0.042 per million input tokens, and the output is free. Since Jev does not generate any text, there is nothing to charge for on the output side. With an LLM, the cost is much higher, and output tokens cost extra.

Personal offline study copy · Content © Outcome School / Amit Shekhar